

Requintu

Documentación Técnica del Proyecto

Versión 2.1 — 18 de septiembre de 2026

1. Resumen del proyecto

Requintu es una aplicación que conecta a viajeros y usuarios con los negocios locales de Colombia. Los usuarios pueden explorar locales a través de un mapa interactivo dividido por departamentos y municipios, consultar el detalle de cada negocio (fotos, promociones, calificaciones, ubicación), publicar contenido en un muro comunitario con avisos en tiempo real y, en el caso de los comerciantes, crear y administrar sus propios locales. Una cuenta de administrador permite gestionar usuarios, locales y publicaciones en un panel dedicado.

El modelo de roles incluye además de turista, comerciante y administrador: comerciante_premium (hasta 5 locales y varias imágenes por local) y alcaldía (publica un local oficial de su municipio que aparece destacado como primer resultado con la insignia «Oficial»). Ambos roles se asignan únicamente por el administrador.

El proyecto está formado por dos clientes y un solo backend:

- Web (SPA React) desplegada en Vercel.
- App nativa Android (Capacitor 8) que empaqueta el mismo frontend (firmada en release para Google Play).
- API REST en Node.js/Express desplegada en Render.

2. Arquitectura general

El sistema sigue una arquitectura de cliente-servidor con API REST y autenticación por tokens JWT, más un canal Server-Sent Events (SSE) autenticado para avisar en vivo de las novedades del muro:

```
Web (Vercel) -----> Backend API (Render) ----> MySQL (TiDB Cloud)
App Android (Capacitor) ----> HTTPS / CORS          ----> Cloudinary (imágenes)
                               SSE autenticado         ----> Brevo (correo)
```

Frontend: React 19 con React Router 7 (SPA). Compilado con Vite 8; se sirve como estático en Vercel con rewrites SPA (configuración en frontend/vercel.json, Root Directory frontend).

App nativa: Capacitor 8 genera el proyecto Android en frontend/android; el WebView carga el mismo dist. Firmada en release (keystore propio) para Google Play.

Backend: Express 4 (módulos ES). Expone /api/*, sirve /uploads y aplica seguridad (helmet, rate limit, CORS, HTTPS). Incluye un canal SSE y una cola de correos persistida.

Base de datos: MySQL 8 administrada (TiDB Cloud) vía mysql2 (pool de conexiones, TLS).

Almacenamiento: las imágenes se suben al backend con Multer (con validación de formato real y cuota diaria por usuario) y se publican en Cloudinary.

Correo: la recuperación de contraseña se encola (tabla cola_emails) y se envía con la API v3 de Brevo, con reintentos y backoff.

Tiempo real: el muro de publicaciones recibe eventos nueva, editada y borrada por SSE; el canal exige autenticación y limita el número de conexiones por cliente y globales para evitar abusos.

3. Stack tecnológico

3.1 Backend (raíz del repositorio)

Tecnología	Uso
Node.js (ESM)	Entorno de ejecución. npm start ejecuta index.js.
Express 4	Servidor HTTP y enrutado REST.
mysql2	Pool de conexiones a MySQL / TiDB Cloud.
jsonwebtoken + bcryptjs	Access tokens JWT (1 h) y hash de contraseñas (bcrypt, 10 rondas).
cookie-parser	Lectura de la cookie httpOnly de refresh token.
multer	Recepción de archivos (solo imágenes JPEG/PNG/WebP, máx. 5 MB, validados por contenido).
cloudinary (SDK 2)	Publicación de imágenes en la nube.
express-validator	Validación y saneamiento de entradas.
express-rate-limit	Límite de peticiones por IP (API y autenticación) y de conexiones SSE.
helmet	Cabeceras de seguridad HTTP (HSTS, CORS cross-origin para recursos).
cors	Lista blanca de orígenes (web + Capacitor).
Server-Sent Events (nativo)	Canal GET /api/eventos (autenticado) para avisos del muro.
nsfwjs + @tensorflow/tfjs	Moderación automática de imágenes NSFW (local, sin terceros).
node:test	Suite de pruebas de integración (npm test, 34 pruebas).
dotenv	Variables de entorno.

3.2 Frontend (carpeta frontend)

Tecnología	Uso
React 19 + Vite 8	SPA y empaquetado de producción.
react-router-dom 7	Enrutado del cliente.
Capacitor 8 (core, cli, android, app)	Empaquetado nativo Android y control del botón «atrás».
Fetch API (SSE con token)	Suscripción del muro a los avisos en tiempo real con Authorization Bearer.
swiper	Carrusel de imágenes (login e inicio).
oxlint	Linter (npm run lint).
Playwright (@playwright/test)	Pruebas e2e (13 pruebas) en Chromium headless.
sharp	Dev: generación de iconos de la app y favicon.

4. Estructura del repositorio

```
proyecto/  
  index.js           # API principal (Express, rutas REST + SSE)  
  db.js             # Pool de conexiones MySQL + health check  
  emailQueue.js     # Cola de correos con reintentos y backoff (Brevo)  
  filtroProfano.js  # Filtro de groserías (leetspeak, repetidos)  
  moderacionImagenes.js # Clasificador NSFW local (nsfwjs) para subidas  
  .env.example      # Plantilla de variables de entorno  
  requintu_dump.sql # Respaldo del esquema/datos (roles, destacado,  
subidas_imagenes)  
  package.json      # Dependencias del backend  
  test/api.test.mjs # 34 pruebas de integración (node:test)  
  Documentacion_Tecnica_Requintu.pdf|docx (v2.1)  
  Informe_de_Pruebas_Requintu.pdf|docx   (v1.4)  
  Manual_de_Uso_Requintu.pdf|docx        (v1.2)  
  Requintu-release.aab|apk (builds firmados)  
  frontend/  
    capacitor.config.json # Config de Capacitor (appId, webDir)  
    index.html            # HTML base (favicon e iconos de la marca)  
    vercel.json           # Config de Vercel (framework, rewrites SPA)  
    public/  
      e2e/                # 13 specs: inicio, privacidad, auth, muro, seguridad, denuncias
```

```

android/          # proyecto Capacitor (firma release en app/build.gradle)
src/
  main.jsx        # Punto de entrada (React Router + AuthProvider)
  config.js       # API_URL según VITE_BACKEND_ORIGIN
  context/AuthContext.jsx # Sesión: login, logout, refresco automático
  pages/         # Home, Login, Register, MapaColombia, Locales,
                  # LocalDetalle, CrearLocal, Publicaciones, Perfil, Admin

```

5. Backend

5.1 Punto de entrada (index.js)

Configura Express, aplica los middlewares globales y registra más de 30 rutas bajo /api. En producción fuerza HTTPS (redirect 301), confía en el proxy (trust proxy), activa HSTS y usa cookies SameSite=None; Secure para permitir el login desde la app nativa. Al arrancar verifica la conexión a la BD y levanta la cola de correos (iniciarColaCorreos()).

5.2 Conexión a la base de datos (db.js)

Crea un pool con mysql2/promise (límite de 5 conexiones por defecto, keep-alive). Si DB_SSL=true (exigido por TiDB Cloud) activa TLS. Exporta checkDatabaseHealth() que valida la conexión con SELECT 1.

5.3 Filtro de groserías (filtroProfano.js)

Normaliza el texto (minúsculas, leetspeak 0-o, 3-e, 4-a, @-a..., caracteres repetidos) y lo compara contra una lista en español. Evita falsos positivos en palabras cortas (longitud mínima). Se aplica con el middleware sinGroserias(campos) a locales, planes, calificaciones y publicaciones.

5.4 Autenticación y autorización

Registro: valida nombre, correo (único) y contraseña (mín. 8). Roles permitidos: turista (por defecto) y comerciante. Los roles comerciante_premium y alcaldia no pueden elegirse en el registro: se asignan por el administrador desde el panel. Contraseña cifrada con bcrypt.

Login: devuelve un access token JWT (1 h) con { id, nombre, rol } y emite una cookie httpOnly con el refresh token.

Refresh token: generado con crypto.randomBytes(64), se guarda hasheado (SHA-256) en la tabla refresh_tokens con expiración de 7 días. Cada refresco rota el token (se revoca el anterior). La cookie se limita a la ruta /api/auth.

Logout: revoca el refresh token y limpia la cookie.

Protección: authMiddleware verifica el JWT (cabecera Authorization: Bearer) y rellena req.user; checkRole([...]) restringe acciones a roles concretos.

5.5 Recuperación de contraseña (Brevo vía cola)

/api/forgot-password genera un token (64 hex, expira en 1 h) guardado hasheado en password_resets. El correo con el enlace se encola en la tabla cola_emails y un procesador lo envía con la API v3 de Brevo con reintentos y backoff. /api/reset-password valida el token (un solo uso), actualiza la contraseña y revoca las sesiones previas.

5.6 Avisos en tiempo real del muro (SSE autenticado)

Endpoint: GET /api/eventos. Desde la v2.1 el canal exige autenticación (Bearer JWT) y aplica límites de conexiones: máx. 5 por IP (429) y 500 globales (503) para evitar abusos (hallazgo F01 de la auditoría de seguridad).

Eventos emitidos: nueva (id, usuario_id y autor al crear una publicación), editada y borrada (id). Incluye la moderación del admin.

Heartbeat: cada 25 s se envía un comentario «: latido» para mantener viva la conexión en proxies (p. ej. Render) y detectar clientes caídos.

Publicación (broadcast): los clientes suscritos se guardan en un Set en memoria; cada evento se serializa y se escribe a todas las conexiones.

El frontend no usa EventSource (que no permite cabeceras): se suscribe con fetch + ReadableStream (helper suscribirseSSE) enviando Authorization: Bearer y con reconexión automática cada 3 s.

5.7 Seguridad

Medida	Implementación
Cabeceras HTTP	helmet + HSTS (365 días) + redirect obligatorio a HTTPS en producción.
CORS	Lista blanca (local, Vercel y Capacitor: capacitor://localhost).
Rate limiting	300 req / 15 min en /api; 10 req / 15 min en login y registro.
Validación de entrada	express-validator en todas las rutas de escritura.
Contenido inapropiado	Middleware sinGroserias.
Subida de archivos	Solo image/jpeg png webp; extensión, MIME y magic bytes comprobados (400 si no es una imagen real), máx. 5 MB.
Cuota de subidas (F02)	Tabla subidas_imagenes con límite diario por rol (turista 10, comerciante 20, comerciante_premium 50, alcaldia 20, admin 100); respuesta 429 al agotarla.
Allowlist imagen_url (F03)	Al crear/editar locales, planes y publicaciones solo se acepta https://res.cloudinary.com/ o /uploads/ (400 en otro caso).
SSE autenticado (F01)	GET /api/eventos exige token y limita conexiones por IP y globales (429/503).
Contraseñas	bcrypt (10 rondas); nunca en texto plano.
Tokens	JWT firmado con JWT_SECRET; refresh tokens hasheados en BD y rotados.
Correo	Cola persistida con reintentos, backoff exponencial y registro del error en BD.

5.8 Subida de imágenes

POST /api/upload (protegido) recibe la imagen con Multer, comprueba sus magic bytes (JPEG, PNG o WebP) rechazando archivos que no sean imágenes reales (400), aplica la moderación NSFW local (moderacionImagenes.js) y la cuota diaria de subidas del usuario (429 al superarla), la sube a Cloudinary (carpeta según entidad) y devuelve la URL pública.

5.9 Cola de correos con reintentos (emailQueue.js)

Persistencia: tabla cola_emails (destinatario, asunto, HTML, estado, intentos, próximo intento, error); se crea con CREATE TABLE IF NOT EXISTS. EncolarCorreo() inserta y dispara el procesamiento; nunca lanza. Un worker envía por Brevo los pendientes con backoff exponencial (1, 2, 4 y 8 min); tras 5 intentos marca fallido. Al iniciar el servidor se reanudan los pendientes (escaneo cada 15 s).

6. API REST (resumen de endpoints)

6.1 Autenticación y perfil

Método	Ruta	Acceso	Descripción
POST	/api/register	Público	Registro (turista/comerciante).

Método	Ruta	Acceso	Descripción
POST	/api/login	Público	Inicio de sesión (JWT + cookie refresh).
POST	/api/auth/refresh	Cookie	Renueva access token (rota refresh).
POST	/api/auth/logout	Cookie	Cierra sesión.
POST	/api/forgot-password	Público	Encola el enlace de recuperación (Brevo).
POST	/api/reset-password	Público	Reestablece contraseña con token de 1 uso.
GET	/api/perfil	Autenticado	Datos del perfil.

6.2 Catálogo

Método	Ruta	Acceso	Descripción
GET	/api/municipios	Público	Lista de municipios.
GET	/api/categorias	Público	Categorías de locales.
GET	/api/departamentos	Público	Departamentos con datos para el mapa.
GET	/api/departamentos/:nombre/municipios	Público	Municipios de un departamento.
GET	/api/eventos	Autenticado	Canal SSE de avisos del muro (nueva/editada/borrada) con límite por IP y global.

6.3 Locales, calificaciones y planes

Método	Ruta	Acceso	Descripción
GET	/api/locales	Público	Listado con filtros y búsqueda (destacados primero).
POST	/api/locales	Comerciante/Premium/Alcaldía/Admin	Crea un local (valida unicidad por municipio; límites por rol: 1 comerciante/alcaldía, 5 premium).
GET	/api/locales/:id	Público	Detalle con calificación promedio y galería.
PUT	/api/locales/:id	Autenticado	Edita (dueño/admin; valida imagen_url).
DELETE	/api/locales/:id	Autenticado	Elimina (dueño/admin).
GET	/api/locales/:id/calificaciones	Público	Reseñas del local.
POST	/api/locales/:id/calificaciones	Autenticado	Puntúa (1-5) y comenta.
GET	/api/locales/:id/planes	Público	Planes/paquetes del local.
POST	/api/locales/:id/planes	Autenticado	Agrega plan (valida imagen_url).
PUT / DELETE	/api/planes/:id	Autenticado	Edita / elimina un plan.

6.4 Subida y publicaciones

Método	Ruta	Acceso	Descripción
POST	/api/upload	Autenticado	Sube imagen a Cloudinary (valida formato real, modera NSFW y cuota diaria por rol).
GET	/api/publicaciones	Público	Muro de publicaciones.
POST	/api/publicaciones	Turista/Admin	Publica contenido (emite evento SSE nueva; valida imagen_url).
PUT / DELETE	/api/publicaciones/:id	Autenticado	Edita / elimina (autor/admin; emite editada/borrada).

6.5 Administración (solo rol admin)

Método	Ruta	Descripción
GET	/api/admin/estadisticas	Totales y distribución por rol (incluye comerciante_premium y alcaldía).
GET	/api/admin/usuarios	Listado con paginación y rol.
PUT	/api/admin/usuarios/:id/rol	Cambia el rol de un usuario (asigna premium/alcaldía).
DELETE	/api/admin/usuarios/:id	Elimina un usuario.

Método	Ruta	Descripción
GET	/api/admin/locales	Listado paginado de locales.
GET	/api/admin/publicaciones	Listado paginado de publicaciones.
DELETE	/api/admin/publicaciones/:id	Modera (elimina) una publicación (emite SSE borrada).

7. Base de datos

MySQL 8 (TiDB Cloud). Esquema principal exportado en requintu_dump.sql:

Tabla	Contenido	Claves principales
usuarios	Nombre, email único, password_hash, rol (admin/comerciante/comerciante_premium/alcaldia/turista), foto_perfil, reset_token y expiración.	PK id_usuario; UNIQUE email
municipios	~1120 municipios de Colombia con su departamento.	PK id_municipio
categorias	13 categorías (restaurante, café, alojamiento, turismo, artesanías, etc.).	PK id_categoria
locales	Nombre, descripción, dirección, teléfono, imagen, destacado (alcaldía), FK a categoría, municipio y propietario.	FKs con ON DELETE SET NULL
locales_imagenes	Galería de varias imágenes por local (comerciante_premium).	FK id_local; CASCADE
calificaciones	Reseñas: puntuación (1-5 con CHECK), comentario, fecha; usuario y local.	FKs con ON DELETE CASCADE
planes	Planes/paquetes del local: título, descripción, precio, fechas, imagen.	FK id_local
publicaciones	Contenido del muro, imagen opcional, fecha, autor.	FK usuario_id
refresh_tokens	Refresh tokens (SHA-256), expiración, revocado, creado_en.	UNIQUE token_hash
password_resets	Tokens de recuperación (hash, expiración, usado).	FK id_usuario
subidas_imagenes	Cuota diaria de subidas: id_usuario y subida_en (límite por rol).	FK id_usuario; CASCADE
cola_emails	Cola de correos: destinatario, asunto, HTML, estado, intentos, próximo intento y error.	PK id; INDEX (estado, proximo_intento)

8. Frontend

8.1 Inicio y rutas (main.jsx, App.jsx)

main.jsx monta React 19 con BrowserRouter y AuthProvider. App.jsx define las rutas públicas (/), las de lectura (/publicaciones, /locales, /locales/:id) y las protegidas (/mapa, /crear-local, /perfil, /admin). También registra el botón atrás de Android vía Capacitor.

8.2 Gestión de sesión (context/AuthContext.jsx)

El access token vive en memoria (estado React); nunca se guarda en localStorage ni en cookies legibles por JavaScript. Al montar la app se llama /api/auth/refresh con la cookie httpOnly. El login usa credentials: incluye para conservar la cookie de refresh (corrección aplicada en la v2.1). Un temporizador renueva el token 5 minutos antes de la expiración. Logout limpia el estado y revoca en el servidor.

8.3 Pantallas principales

Página	Función
Home	Portada con carrusel (swiper) y accesos a secciones.
Login / Register	Acceso con carrusel decorativo y diseño responsive.
MapaColombia	Mapa SVG de departamentos con zoom/pan propio; al tocar un departamento se muestran sus municipios.

Página	Función
Locales	Filtros por departamento, municipio y categoría; tarjetas con estrellas y promociones. Los locales oficiales (alcaldía) aparecen primero con insignia «Oficial».
LocalDetalle	Fotos, descripción, mapa de Google, planes, galería de calificaciones y edición (dueño).
CrearLocal	Formulario + vista previa del mapa; alta de local con límites por rol y galería de imágenes para premium.
Publicaciones	Muro con avisos en tiempo real vía SSE autenticado (helper suscribirseSSE con fetch + token).
Perfil	Datos personales y edición.
Admin	Panel: estadísticas y pestañas Usuarios/Locales/Publicaciones con paginación; asigna roles premium y alcaldía.

8.4 Configuración de entorno

config.js lee VITE_BACKEND_ORIGIN (variable de compilación) y construye API_URL; si no está definida usa http://localhost:3000. La conexión SSE usa el mismo origen. En la app nativa se compila apuntando a https://requintu-1.onrender.com.

9. App nativa Android (Capacitor)

Configuración: frontend/capacitor.config.json con appId com.requintu.app, appName Requintu y webDir dist. Proyecto nativo generado con npx cap add android; el web build se copia con npx cap sync android. Compilación: Gradle 8.14.3 (JDK 21), compileSdk/targetSdk 36, minSdk 24.

Firma release: keystore propio (frontend/android/requintu-release.jks) leído desde keystore.properties por app/build.gradle; ambos excluidos de git. Artefactos entregables en la raíz: Requintu-release.aab (Google Play, verificado con apksigner), Requintu-release.apk, iconos para la ficha (Requintu-icone-play-512.png, Requintu-feature-graphic-1024x500.png). Marca: iconos propios en todas las densidades, fondo #12283D y favicon con el logo.

Interacción nativa: botón «atrás» gestionado con @capacitor/app. Backend compatible: CORS admite capacitor://localhost y las cookies en producción usan SameSite=None para el WebView.

10. Despliegue

Backend (Render): https://requintu-1.onrender.com (npm start, NODE_ENV=production, variables según .env.example, health check de la BD, SSE con heartbeat 25 s).
Frontend (Vercel): https://requintu.vercel.app (push a main = despliegue automático; Root Directory frontend; VITE_BACKEND_ORIGIN=https://requintu-1.onrender.com).
Base de datos (TiDB Cloud): MySQL en la nube, host gateway01.us-east-1.prod.aws.tidbcloud.com, puerto 4000, SSL rejectUnauthorized: true.

Nota: la PWA (service worker) se desactivó a propósito (SW auto-destructivo) para que la web cargue siempre la versión más reciente.

11. Variables de entorno

NODE_ENV, JWT_SECRET, DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, DB_SSL, CORS_ORIGINS, RATE_LIMIT_API_MAX, RATE_LIMIT_AUTH_MAX, BREVO_API_KEY, BREVO_SENDER_EMAIL, EMAIL_MAX_INTENTOS (por defecto 5), APP_URL, PORT, CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET, LIMITE_UPLOADS_<ROL> (override de la cuota por rol),
VITE_BACKEND_ORIGIN (frontend, en compilación).

Advertencia: el archivo .env real no debe subirse a git (está ignorado).

12. Ejecución local

Backend: npm install ; cp .env.example .env ; npm run dev | npm test (34 pruebas)

```
Frontend: cd frontend; npm install; npm run dev; npm run lint; npm run build;
          npx playwright install chromium; npm run test:e2e (13 pruebas)
Android:  cd frontend; VITE_BACKEND_ORIGIN=https://requintu-1.onrender.com npm run build;
          npx cap sync android; cd android; ./gradlew assembleDebug; ./gradlew
          bundleRelease
```

13. Seguridad y buenas prácticas aplicadas

- Contraseñas con bcrypt y tokens JWT con expiración corta, + refresh tokens rotatorios en cookie httpOnly.
- Cabeceras de seguridad (helmet/HSTS), HTTPS forzado y CORS por lista blanca.
- Rate limiting por IP en API y autenticación, y límites de conexiones SSE.
- Validación de entradas y saneamiento en todas las rutas de escritura (incluida la allowlist de imagen_url).
- Filtro de lenguaje inapropiado para contenido generado por usuarios.
- Moderación NSFW de imágenes con clasificador local (sin enviar fotos a terceros).
- Cuota diaria de subidas por usuario según rol.
- Restricción de subidas por formato real (magic bytes), MIME y tamaño.
- Reintentos y backoff en el envío de correos mediante cola persistida.
- Variables sensibles únicamente en variables de entorno.
- Verificación de unicidad del nombre de un local dentro de su municipio.

14. Pruebas automatizadas

Backend (node:test): npm test ejecuta test/api.test.mjs con 34 pruebas de integración contra la base de datos real: autenticación, roles (turista, comerciante, comerciante_premium con límite de 5 locales, alcaldia con local destacado), CORS, CRUD de publicaciones, filtros de locales, validación de imágenes (magic bytes), cuotas de subida (F02), allowlist de imagen_url (F03), SSE autenticado (F01), recuperación de contraseña y rotación/revocación de sesiones.

Frontend (Playwright): npm run test:e2e ejecuta 13 pruebas e2e en Chromium headless: home, buscador de locales, muro, política de privacidad, rutas protegidas, registro/login/logout, persistencia de sesión al recargar (cookie httpOnly), mapa con sesión, aviso en tiempo real del muro, XSS en muro y comentarios, y denuncias.

El detalle caso a caso está en el Informe de Pruebas (v1.4).

15. Mejoras futuras sugeridas

- Publicar la app en Google Play (subir el Requintu-release.aab).
- Moderación manual de locales antes de publicarlos (flujo de aprobación).
- Migraciones versionadas del esquema (hoy se respalda con dump).
- Panel de la cola de correos en el admin.
- Notificaciones push nativas (FCM) para la app Android.
- Geolocalización en tiempo real y pagos en línea para planes premium.

16. Glosario de rutas del servidor

```
/api/* – API REST y canal SSE autenticado (/api/eventos).
/uploads/* – archivos estáticos locales (legacy de imágenes).
Cualquier otra ruta en web la resuelve el SPA (rewrites de Vercel/Netlify).
```

Fin de la documentación técnica.